

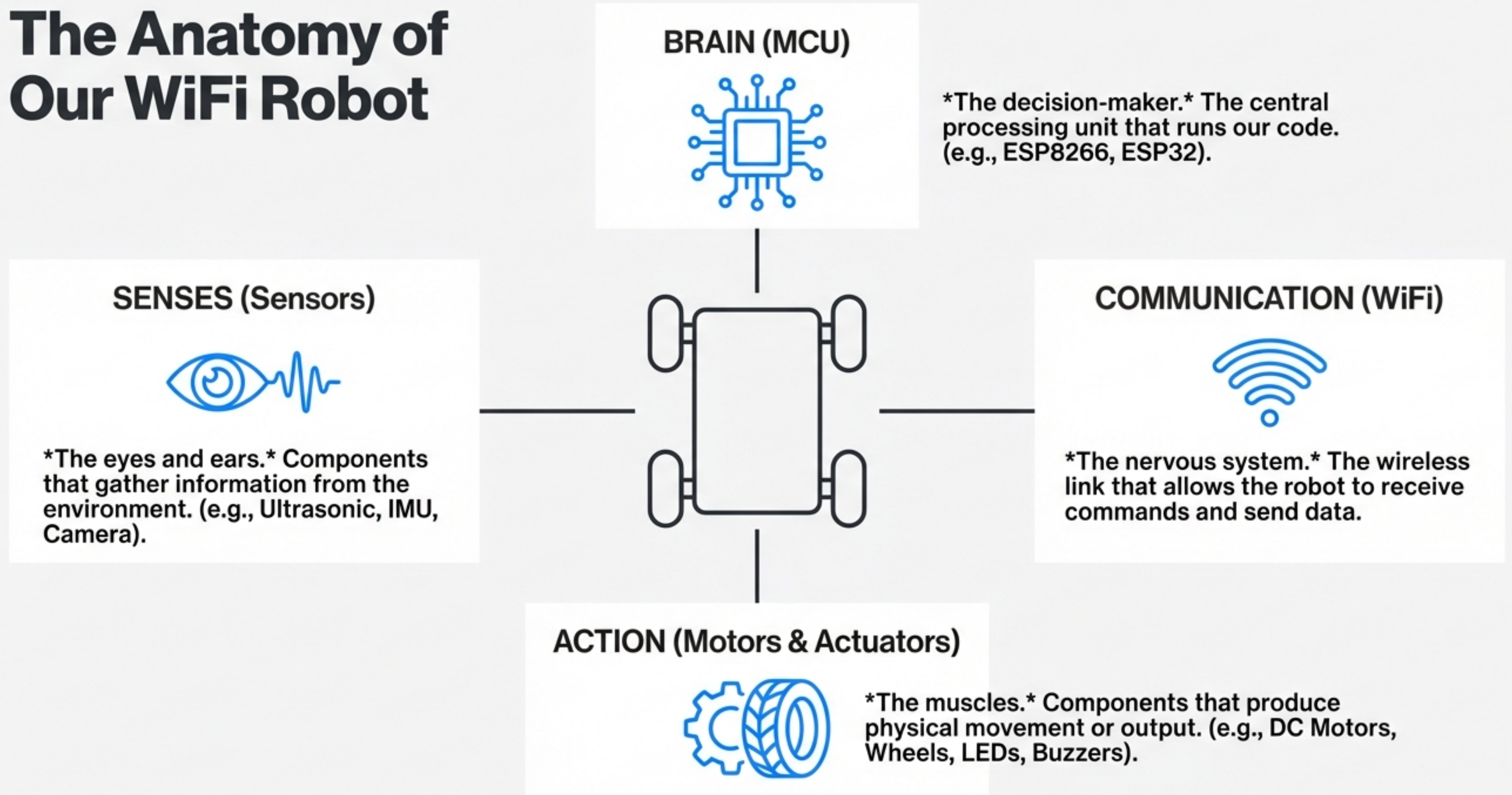
# The Evolution of a WiFi Robot

A Code-Driven Journey from Remote Control to Autonomous Intelligence





# The Anatomy of Our WiFi Robot

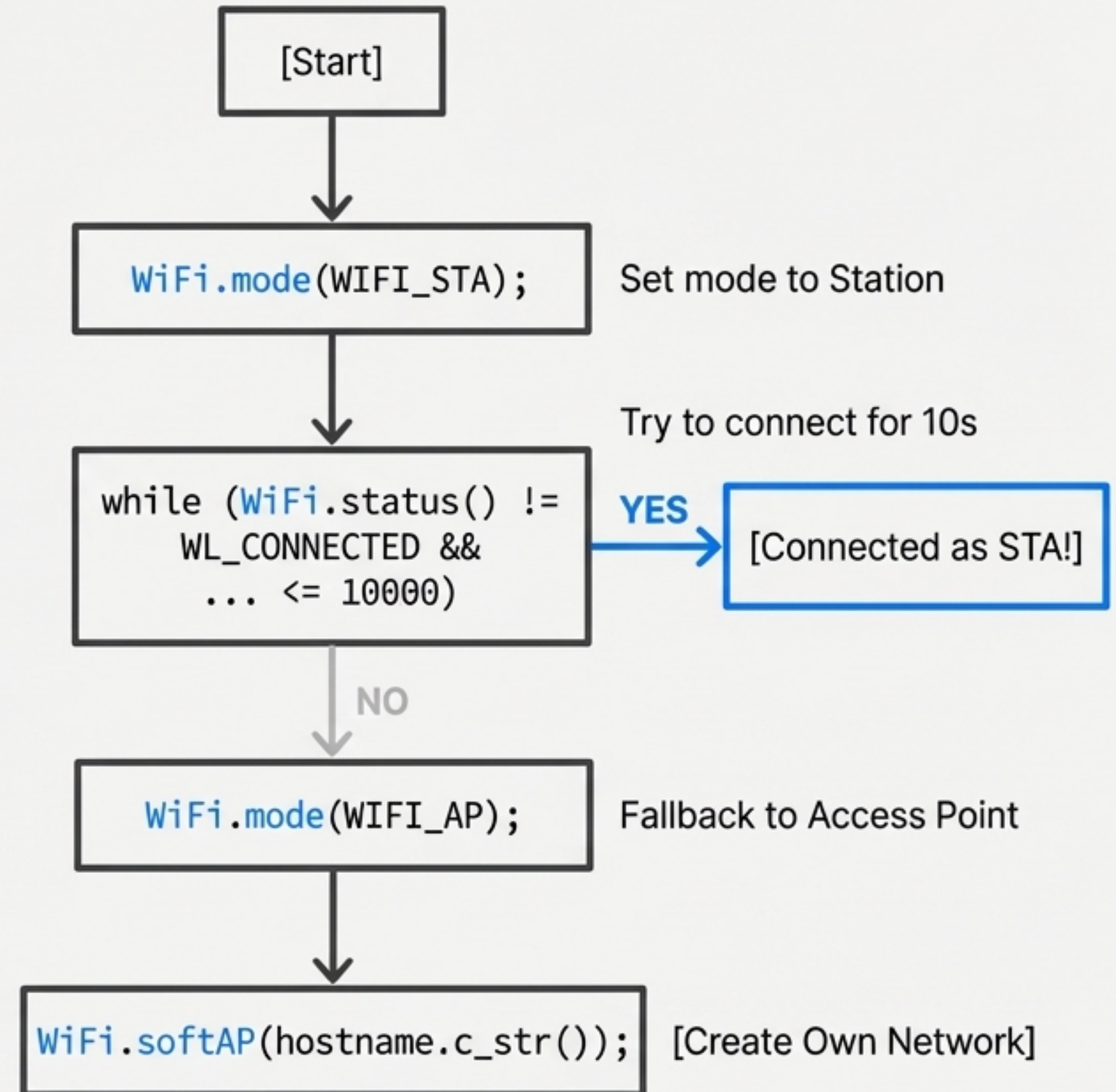




# The Unsung Hero: The WiFi Connection Manager

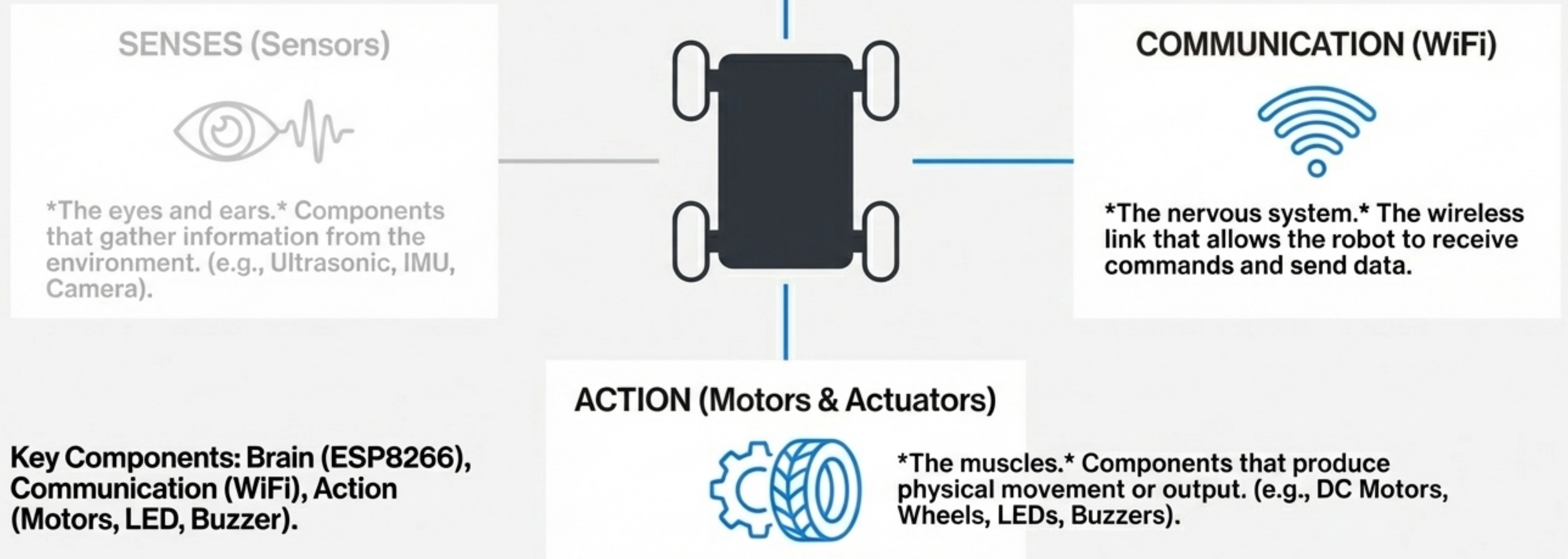
All our robots share a brilliant piece of code for rock-solid connectivity. This dual-mode logic ensures the robot is always accessible.

1. **Attempt Station (STA) Mode:** The primary goal. The robot tries to connect to a pre-defined home network. This is ideal for integration and stability.
2. **Fallback to Access Point (AP) Mode:** If connecting to the STA network fails after a 10-second timeout, the robot creates its own WiFi network. This guarantees you can always connect to it directly, even in a new environment.





# Level 1: Taking the First Step with Direct Remote Control





# How It Listens: The HTTP Server Command Loop

## Receiving Command

### 1. Listening for Requests

An ESP8266WebServer object continuously checks for incoming connections. When a request like `http://<IP>/?State=F` arrives, it extracts the argument's value.

```
// Create server on port 80
ESP8266WebServer server(80);

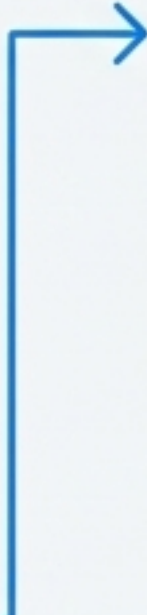
void loop() {
  // Listen for HTTP requests
  server.handleClient();

  // Extract "State" argument from URL
  command = server.arg("State");
}
```

## Executing Command

### 2. Parsing and Acting

An if/else if chain compares the command string and calls the corresponding function to move the motors.



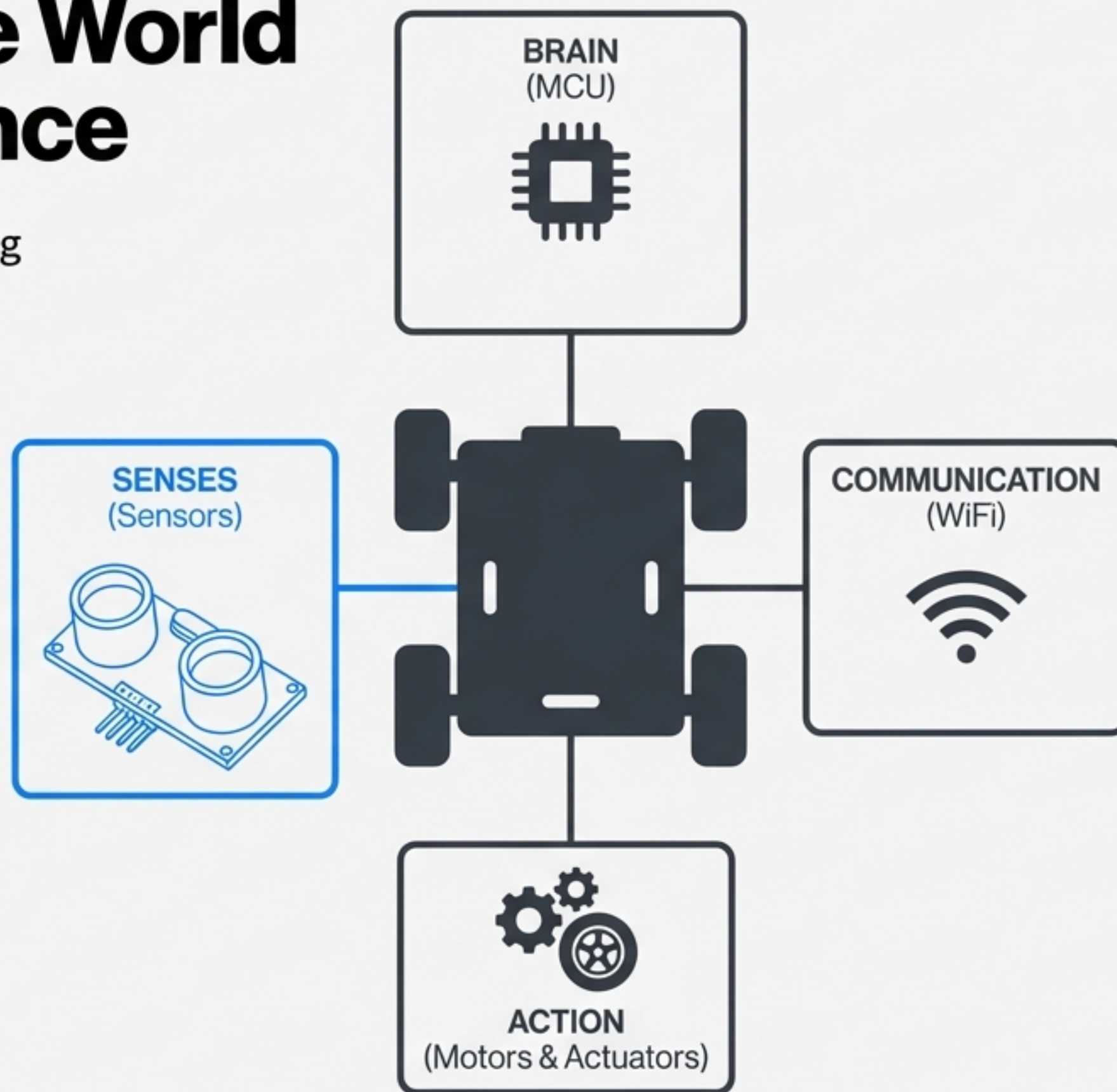
```
if (command == "F") Forward();
else if (command == "B") Backward();
else if (command == "R") TurnRight();
else if (command == "L") TurnLeft();
else if (command == "S") Stop();
```



# Level 2: A Sense of the World with Obstacle Avoidance

We give the robot “eyes.” Instead of waiting for commands, it can now perceive its environment and make a simple decision on its own: “Is there something in front of me?”

**The Upgrade:** The critical addition is an HC-SR04 Ultrasonic Sensor, which allows the robot to measure distance.





# From Sensor Data to Autonomous Decision



```
int getDistance() {  
    // Send 10 microsecond pulse  
    digitalWrite(trigPin, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(trigPin, LOW);  
  
    // Measure the echo return time  
    echoTime = pulseIn(echoPin, HIGH);  
  
    // Convert time to distance  
    return echoTime / 58.26;  
}
```



## 2. Make the Decision

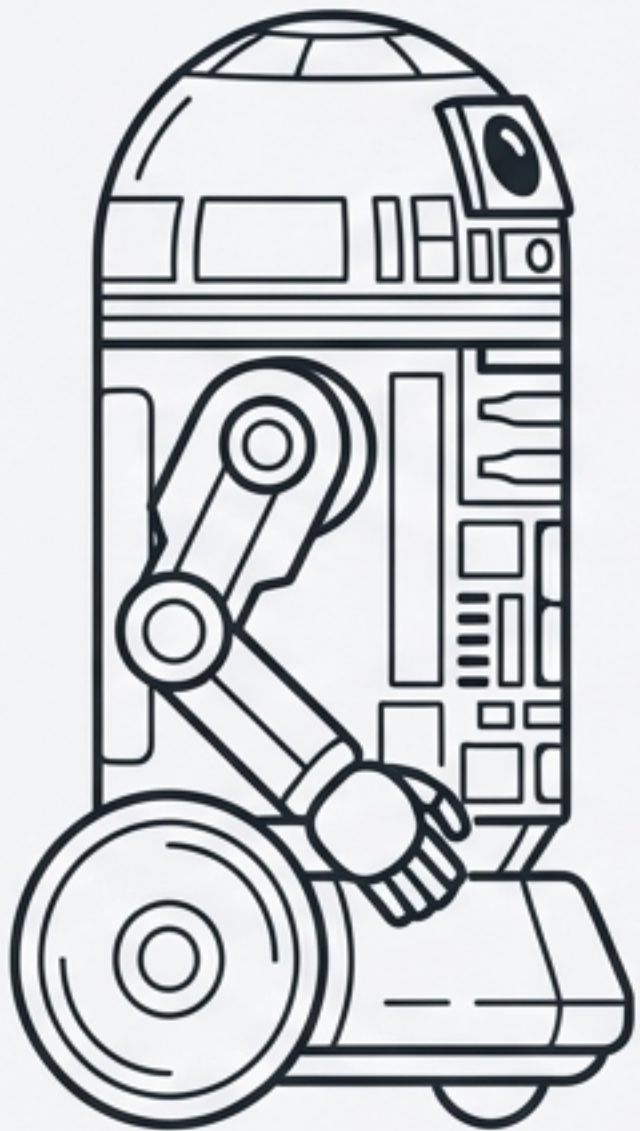
In the main loop, if the measured distance is below a threshold (distanceHold), the robot executes an avoidance maneuver.

```
// Get the latest distance reading  
distanceCm = getDistance();  
  
// Is an object too close?  
if(distanceCm <= distanceHold){  
    // If yes, turn right to avoid  
    TurnRight();  
    delay(400);  
}
```



## Level 2.5: From Avoiding to Following

With a minor tweak to the code, we change the robot's behavior entirely. Instead of simply reacting when an object is 'too close,' the robot now actively tries to maintain a *specific* distance from an object.



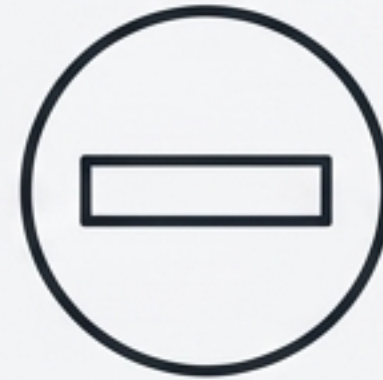
**Too Close**



Move Backward

```
if(distanceCm < distanceKeep)  
-> Backward();
```

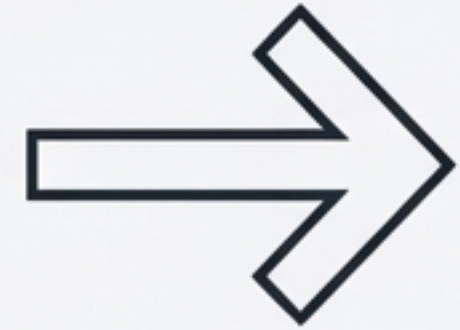
**Just Right**



Stop

```
if(distanceCm >= distanceKeep &&  
distanceCm <= (distanceKeep+3))  
-> Stop();
```

**Too Far**



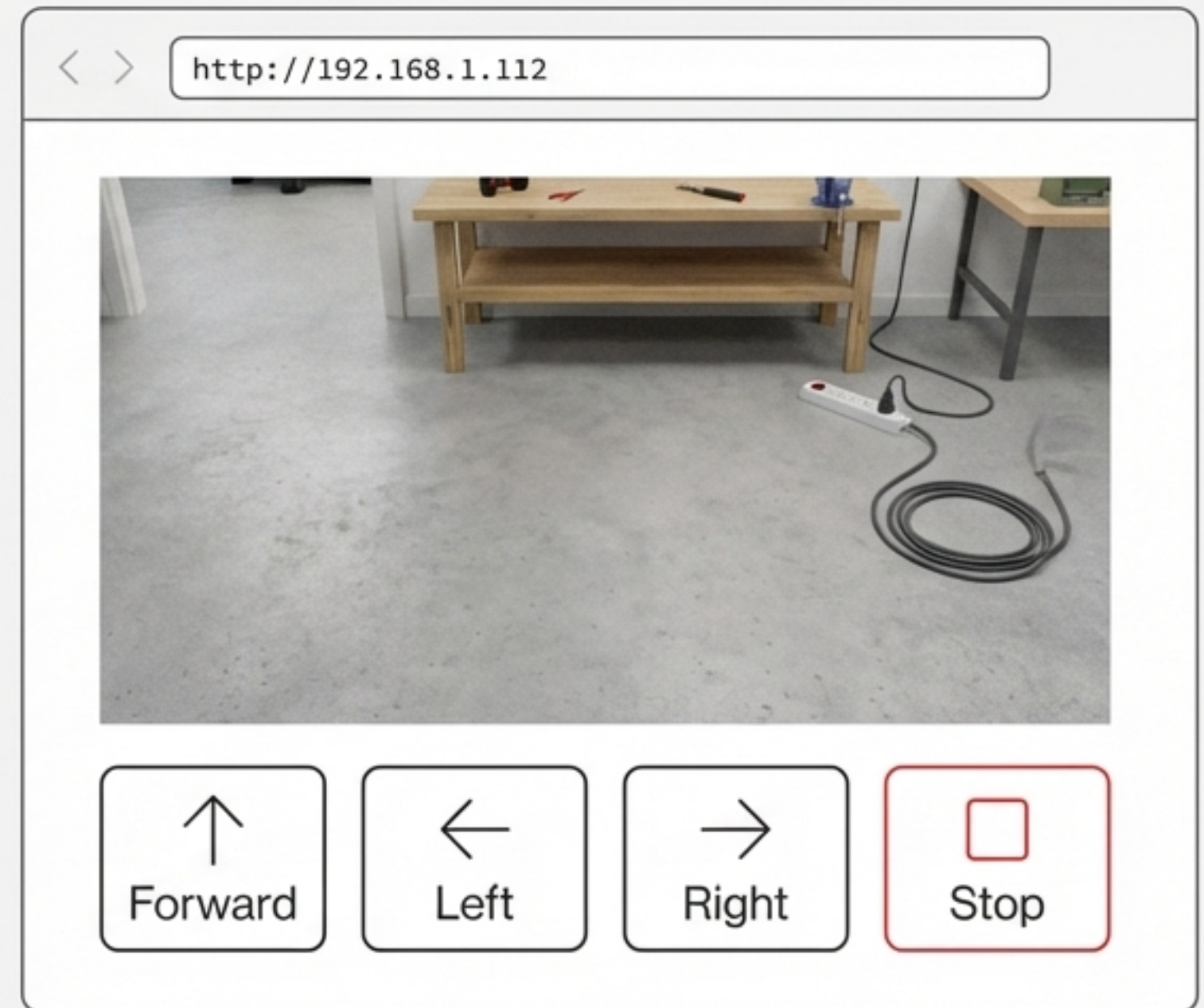
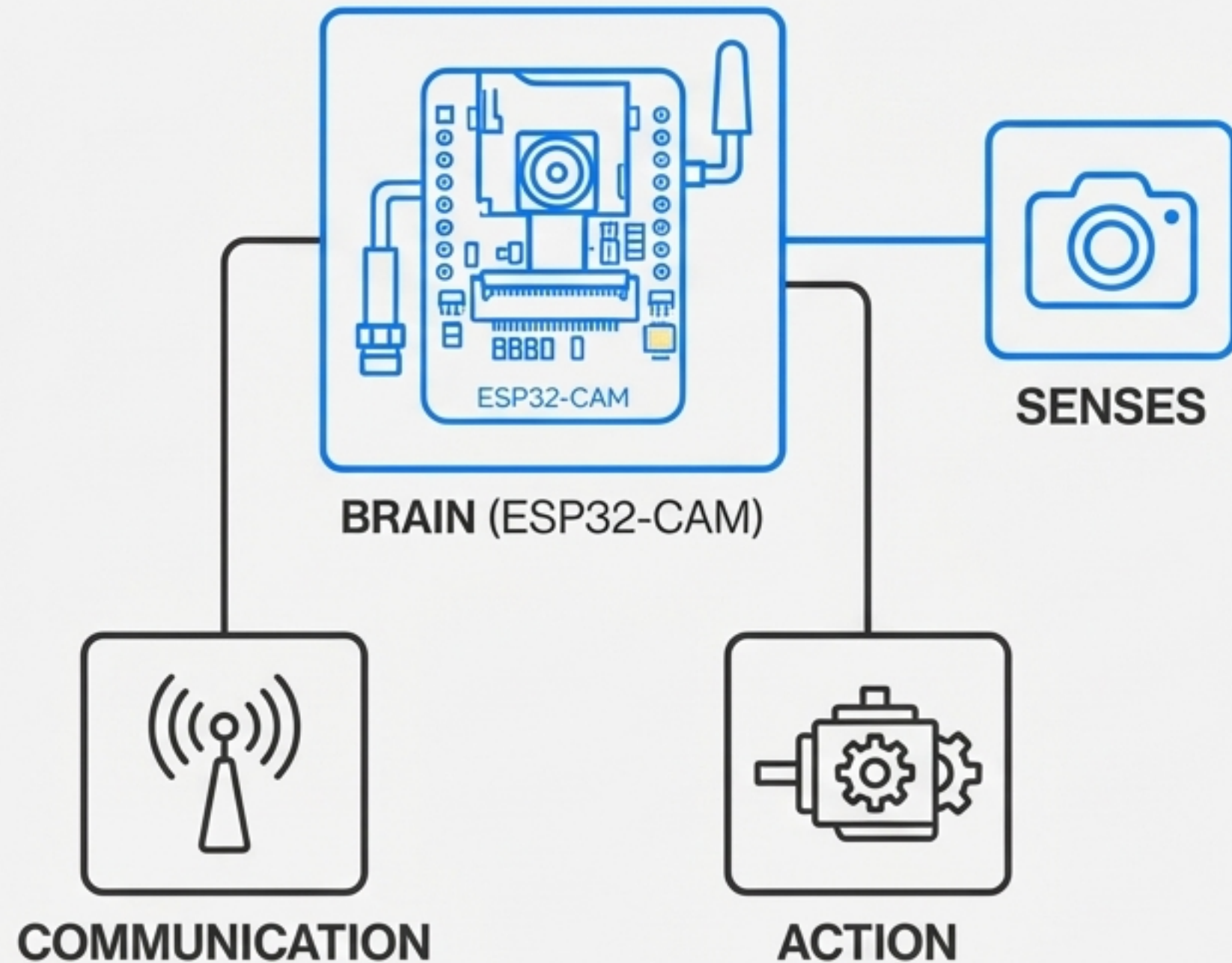
Move Forward

```
if(distanceCm > (distanceKeep+3))  
-> Forward();
```



# Level 3: A New Perspective with Live Video Streaming

We're giving the robot a live camera feed, turning it into mobile eye. This This requires a more powerful brain, the ESP32-CAM, which is specifically designed to handle the demands of image processing and streaming.





# Initializing the Eye: Camera and Server Setup

Getting the camera running is a three-step configuration process. While the code looks complex, it's mostly about telling the ESP32 how the camera is connected and what settings to use.

## Step 1: Define Pins

A block of `#define` statements maps the physical pins of the ESP32 to the camera module's functions.

```
#define PWDN_GPIO_NUM 32
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
// ... more pins
```

## Step 2: Configure Camera

A `camera\_config\_t` structure is filled with key parameters like pixel format and frame size to define video quality.

```
camera_config_t config;
config.pixel_format = PIXFORMAT_JPEG;
config.frame_size = FRAMESIZE_CIF;
// ... more settings
```

## Step 3: Start Server

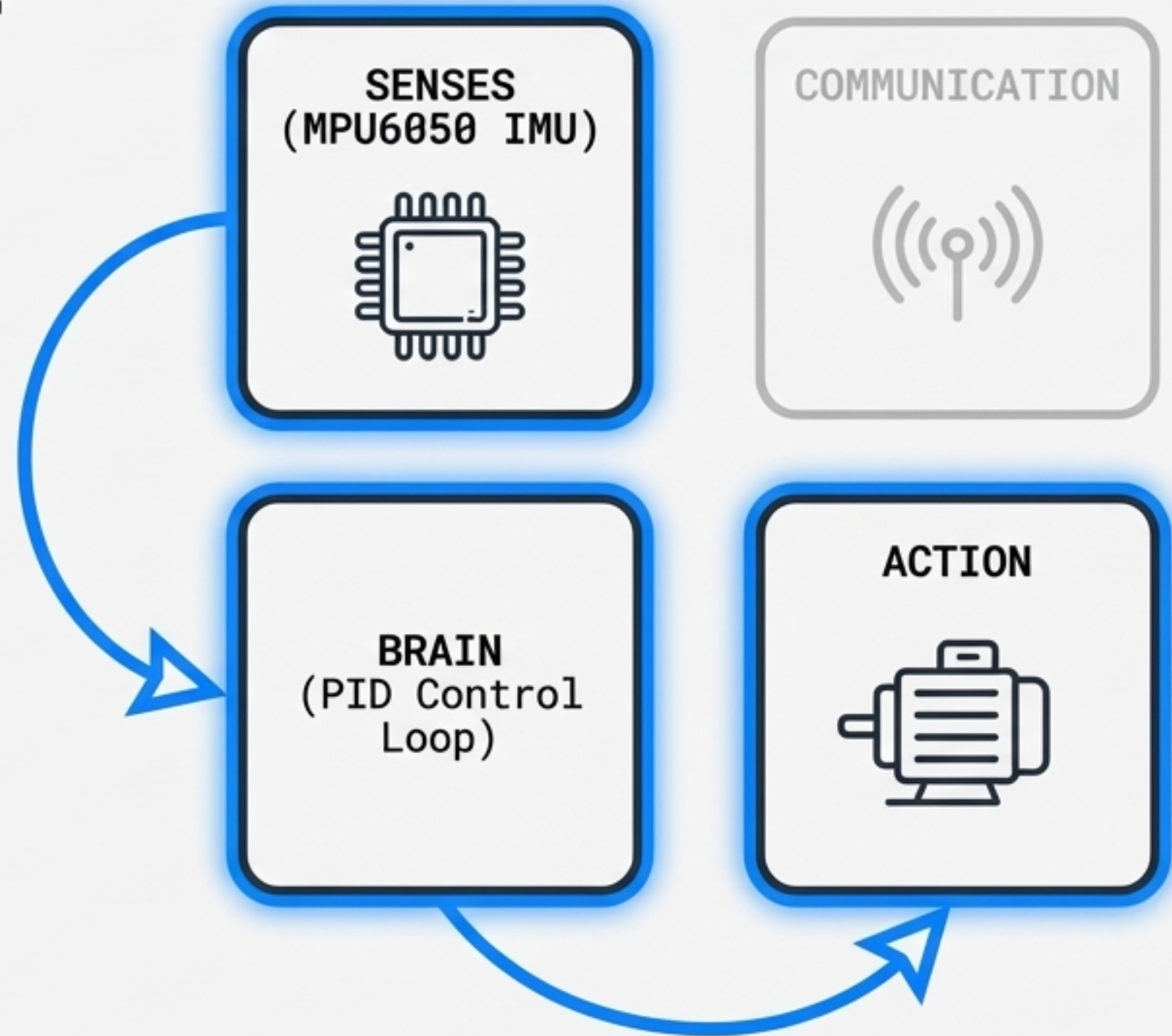
Finally, the camera is initialized and a single function call starts the web server that streams the video feed.

```
esp_err_t err = esp_camera_init(&config);
startCameraServer();
```



# Level 4: Mastering Gravity with the Self-Balancing Robot

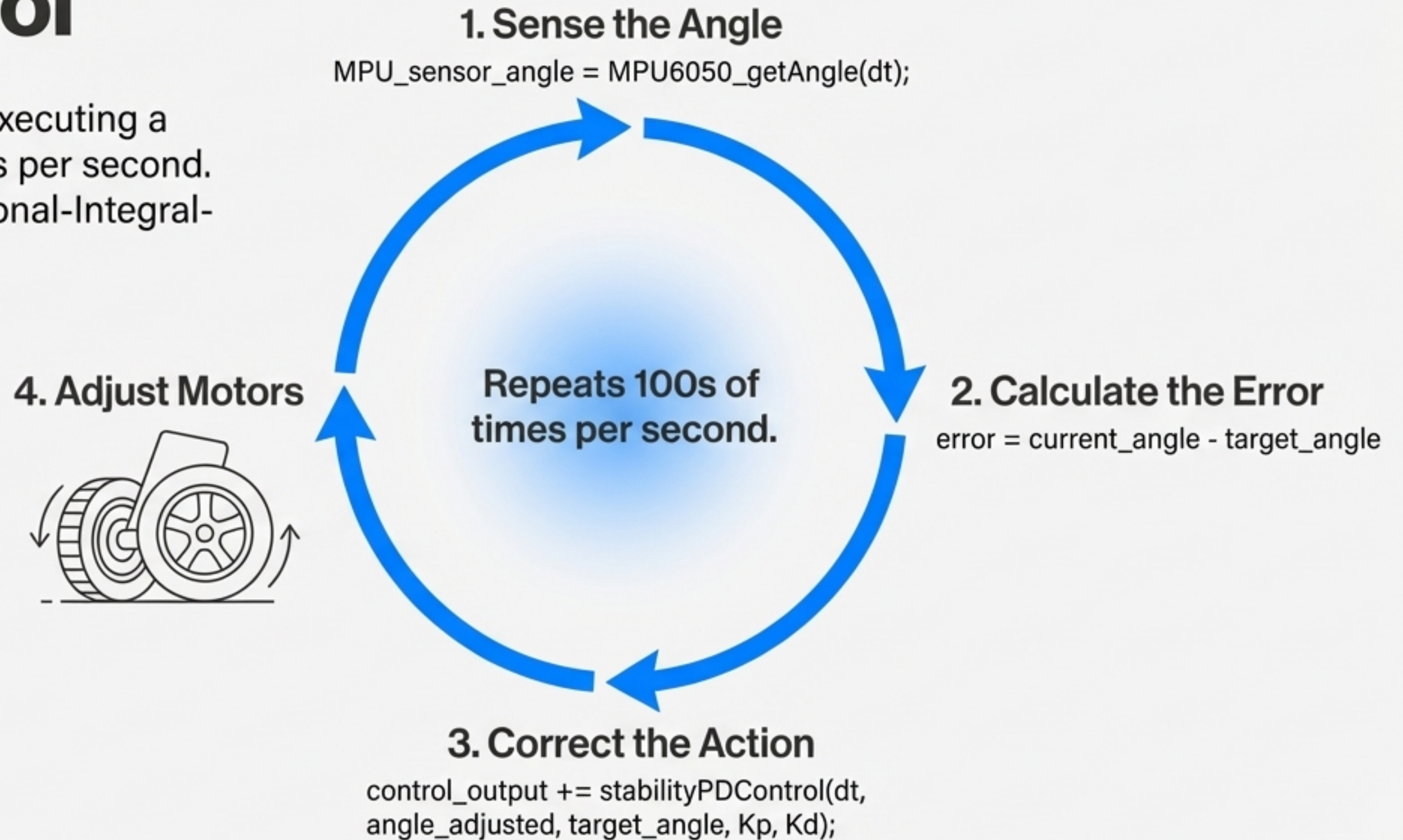
The pinnacle of our skill tree. This robot actively defies gravity by constantly measuring its tilt and adjusting its wheels. This isn't a pre-programmed action; it's a dynamic, real-time feedback loop.





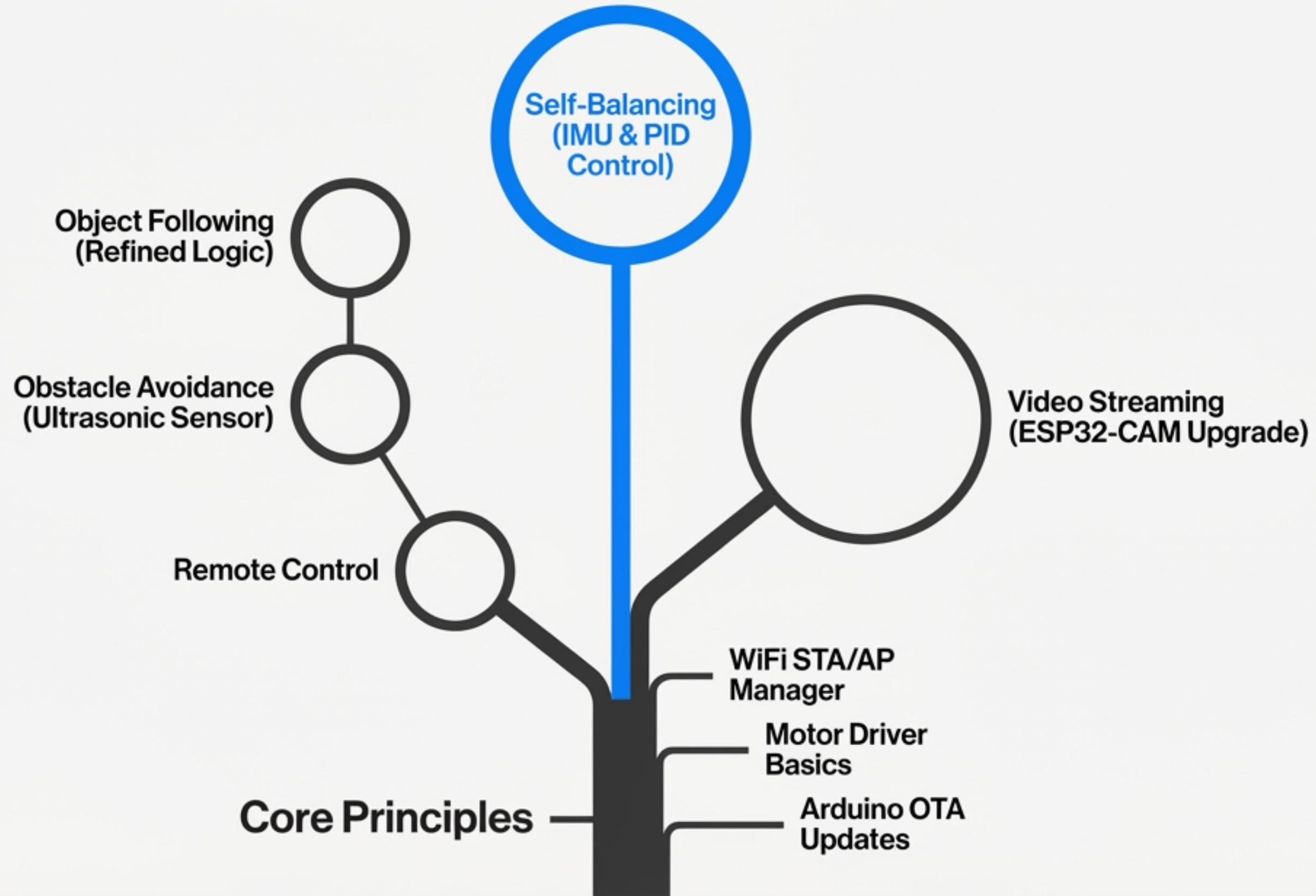
# The Constant Dance of PID Control

The robot stays balanced by executing a simple loop, hundreds of times per second. This is the role of the Proportional-Integral-Derivative (PID) controller.





# Your Robotics Skill Tree





# Engineering Principles for Your Projects



## Start Simple, Iterate Often

Our journey began with a basic remote-controlled chassis. Each new feature was a single, manageable addition. Build a solid foundation, then add complexity one layer at a time.



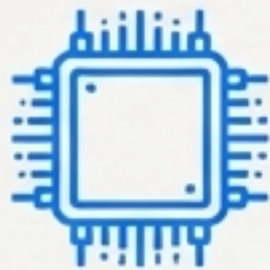
## Abstract Complexity

Functions like `Forward()` or `getDistance()` make the main `loop()` clean and readable. Encapsulate logic into reusable blocks to manage complexity.



## Plan for Failure

The STA/AP WiFi manager is a perfect example of robust design. Anticipating that a WiFi connection might fail and providing a fallback makes your project more reliable and user-friendly.



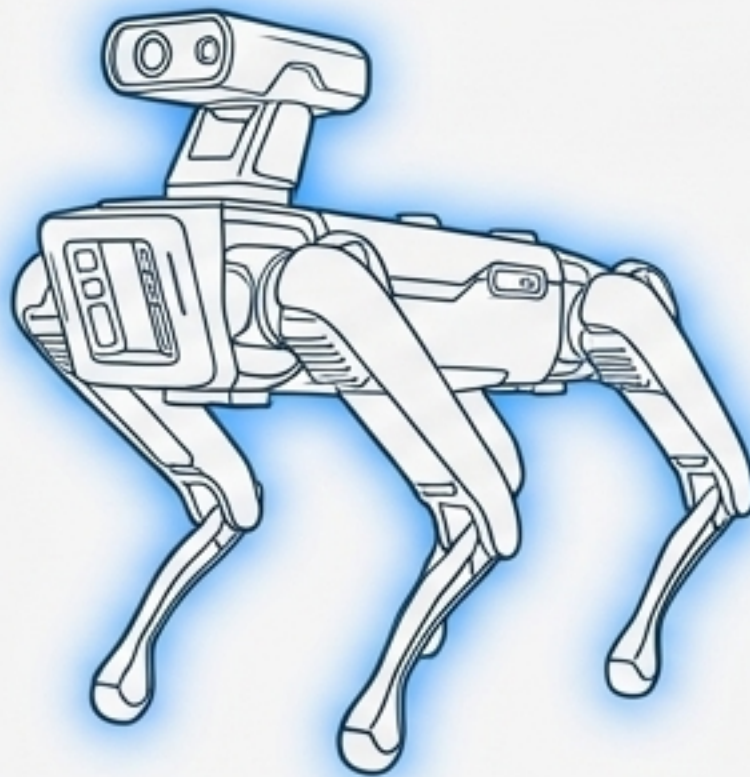
## The Right Tool for the Job

An ESP8266 is perfect for simple I/O and web commands. But data-heavy tasks like video streaming or high-frequency PID loops demand the power of an ESP32. Match your hardware to your ambition.



# Build Your Own Legend

**Where will your journey take you next?**



You've seen the building blocks. Now, combine them.

Can you build a self-balancing robot that streams video of its path?

What about an object-follower that uses its camera to track color instead of distance?

The skill tree is yours to grow.



Find all source code and documentation at:  
[[github.com/example/robot-projects](https://github.com/example/robot-projects)]